Integración de Servicios Cloud GCP

Contenedores: Docker, Docker Compose y Kubernetes

Semana 14 - Sesión 27

Rel Guzman



Revisión: lo que vimos en la Sesión 26

Balanceador de carga

Recibe a todos los clientes en una sola dirección y los reparte entre varios servidores iguales.

CDN

Guarda cerca del cliente una copia de los archivos que son iguales para todos.

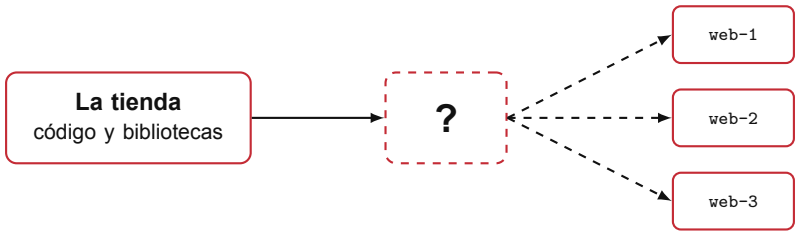
Contenedor

Un paquete con tu programa y todo lo que necesita para funcionar.

Responde

1. Con 3 servidores y round robin, ¿a cuál va el cliente 5?
2. Si `web-2` no responde, ¿qué hace el balanceador?
3. ¿Por qué el carrito de Ana no va en la CDN?

Revisión: la pregunta de hoy



1. Docker	2. Docker Compose	3. Kubernetes
¿Cómo se arma un paquete con la tienda y todo lo que necesita?	¿Cómo se encienden varios contenedores juntos?	Si una copia de la tienda se cae, ¿quién enciende otra?

Logro de la sesión

Al finalizar, podrás explicar con tus palabras qué es un **Dockerfile** y para qué sirven **Docker**, **Docker Compose** y **Kubernetes**. Además, encenderás en tu computadora una **API** hecha con **Python** y **Flask**.

Saberes previos

- En la Sesión 11, antes de encender el programa de Python, instalaste algo con `pip`. ¿Qué era y para qué servía?
- Cuando escribes `localhost:8080` en el navegador, ¿qué indica el 8080?
- Si se apaga uno de los tres servidores de la tienda, ¿quién debería encender otro?

Utilidad de esta sesión

Para tener varias copias iguales de un programa, primero hay que **empaquetarlo** con todo lo que necesita. Hoy verás cómo se arma ese paquete, cómo se encienden varios juntos y cómo se mantienen encendidos. En la práctica encenderás una API y verás qué necesita para funcionar.

Contenedores

Docker, Docker Compose y Kubernetes

Docker: imagen y contenedor

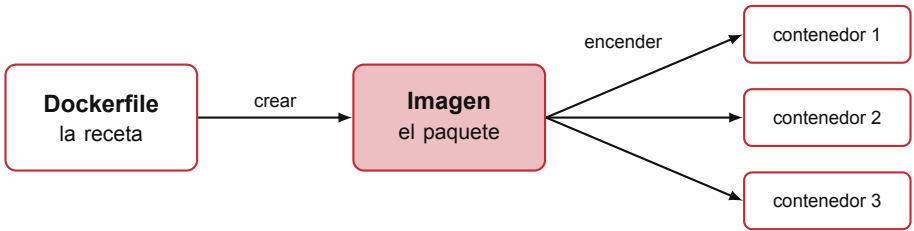


Imagen	Contenedor
El paquete guardado: el programa y todo lo que necesita. Está apagado.	Una imagen **encendida**. De una imagen puedes encender muchos contenedores iguales.

Docker es el programa que crea las imágenes y enciende los contenedores.

El Dockerfile de la tienda

```
FROM node:22-alpine
WORKDIR /app
COPY package.json server.js ./
CMD ["node", "server.js", "3001"]
```

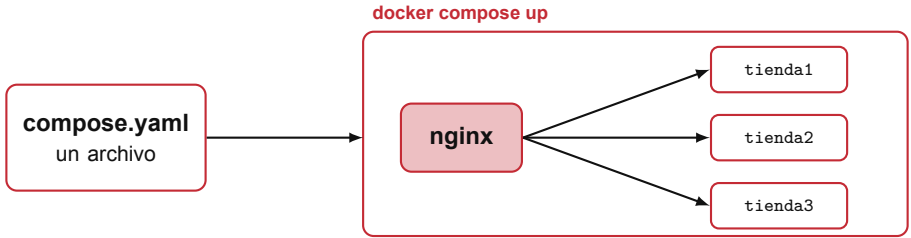
Línea	En palabras
`FROM`	Empieza con Node.js ya instalado
`WORKDIR`	Trabaja en la carpeta `/app`
`COPY`	Copia los archivos de la tienda
`CMD`	Al encender, ejecuta la tienda

```
docker build -t tienda .          # crea la imagen "tienda"
docker run -p 3001:3001 tienda    # enciende un contenedor
```

Lo importante

No instalaste Node.js en tu computadora: viene dentro de la imagen.

Docker Compose: varios contenedores a la vez

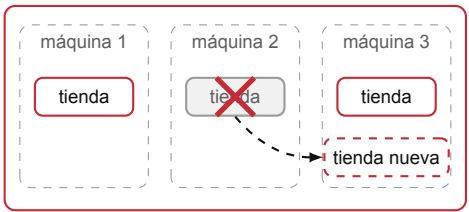


Docker Compose

Describe en **un archivo** todos los contenedores de tu aplicación y los enciende juntos con **un comando**.

A mano	Con Docker Compose
Instalar Node.js y Nginx	Solo tener Docker
4 terminales, 4 comandos	1 comando: `docker compose up`

Kubernetes: contenedores en muchas máquinas



¿Por qué no basta Compose?

Compose usa **una sola máquina**. Si esa máquina se apaga, se apaga todo.

Kubernetes

Tú le dices: «quiero **3 copias** de la tienda». Kubernetes las reparte en varias máquinas y se encarga de que **siempre haya 3**.

Si se cae una copia

Kubernetes lo nota y enciende otra, sin que tú hagas nada.

En Google Cloud

Se llama **Google Kubernetes Engine** (GKE).

Las tres herramientas

	Docker	Docker Compose	Kubernetes
Qué hace	Crea la imagen y enciende un contenedor	Enciende varios contenedores juntos	Mantiene muchas copias encendidas
Dónde	Una máquina	Una máquina	Varias máquinas
Ejemplo	La imagen de la tienda	3 tiendas + Nginx en una máquina	La tienda en producción

En una frase

Docker **empaqueta**, Docker Compose **junta** y Kubernetes **mantiene**.

Práctica

La API de Café Nube, con Python y Flask en tu computadora

Práctica: la API de Café Nube

Caso de estudio

Café Nube, una cafetería de barrio, quiere mostrar su carta en internet. Primero necesita una **API**: un programa que no muestra páginas, sino que **entrega datos** a otros programas.

```python
import os
from flask import Flask, jsonify
app = Flask(__name__)

MENU = [
    {"nombre": "Americano", "precio": 8},
    {"nombre": "Capuchino", "precio": 10},
    {"nombre": "Mocaccino", "precio": 12},
]

@app.get("/menu")
def menu():
    return jsonify(MENU)

puerto = int(os.environ.get("PUERTO", 8080))
app.run(port=puerto)
```

Línea	En palabras
`from flask import ...`	Usa Flask, una biblioteca para hacer APIs
`MENU = [...]`	La carta: nombre y precio en soles
`@app.get("/menu")`	Si alguien pide `/menu`…
`return jsonify(MENU)`	…le responde la carta en JSON
`os.environ.get(...)`	Lee la variable `PUERTO`; si no existe, usa 8080
`app.run(port=puerto)`	Enciende la API y espera pedidos

JSON: texto con datos que otro programa puede leer:
`{"nombre":"Americano","precio":8}`

Práctica: preparar y encender

macOS / Linux

```
# 1. Crea el entorno virtual
python3 -m venv .venv
# 2. Enciende la API (prueba 1)
.venv/bin/python api.py
# 3. Instala lo de requirements.txt
.venv/bin/python -m pip install -r
    requirements.txt
# 4. Enciéndela otra vez
.venv/bin/python api.py
```

Windows (PowerShell)

```
# 1. Crea el entorno virtual
python -m venv .venv
# 2. Enciende la API (prueba 1)
.venv\Scripts\python api.py
# 3. Instala lo de requirements.txt
.venv\Scripts\python -m pip install -r
    requirements.txt
# 4. Enciéndela otra vez
.venv\Scripts\python api.py
```

Comprueba en el navegador: `http://localhost:8080/menu`

```
[{"nombre":"Americano","precio":8},{"nombre":"Capuchino","precio":10},
 {"nombre":"Mocaccino","precio":12}]
```

Carpeta `Sesion27/practica_cafe_nube/`. Necesitas Python 3.10 o más nuevo. La guía completa está en el `README.md`.

Práctica: seis pruebas

#	Qué hacer	Qué observar
1	Paso 2: enciende la API **antes** de instalar `requirements.txt`	¿Qué error sale? ¿Qué le falta a la API?
2	Abre `localhost:8080/menu` o, en otra terminal, `curl localhost:8080/menu`	¿Qué responde? ¿Qué anota la terminal de la API?
3	Abre `localhost:8080/postres`	¿Qué código anota la terminal?
4	En otra terminal, enciende la API otra vez	¿Qué mensaje sale? ¿Por qué?
5	`PUERTO=9090 .venv/bin/python api.py`	¿Ahora sí enciende? Abre `localhost:9090/menu`
6	Agrega un café a `MENU` y recarga `/menu`	¿Aparece? ¿Y después de reiniciar la API?

En Windows: `curl.exe` en la prueba 2; en la prueba 5, primero `$env:PUERTO = "9090"` y luego `.venv\Scripts\python api.py`.

La idea

Además de su código, la API necesita **Python**, **Flask** (lo que dice `requirements.txt`) y **un puerto libre**. Reto en el `README.md`: la ruta `/menu/<nombre>`.

Cierre: preguntas para comprobar

Responde con tus palabras

1. ¿Qué es un Dockerfile y qué se obtiene con `docker build`?
2. En el Dockerfile de la tienda, ¿por qué no tuviste que instalar Node.js en tu computadora?
3. ¿Qué hace Kubernetes que Docker Compose no hace?
4. En la práctica, ¿por qué falló la API antes de instalar `requirements.txt`?
5. ¿Por qué no pudiste encender dos veces la API en el puerto 8080? ¿Cómo lo resolviste sin cambiar el código?

Idea final

Docker **empaqueta**, Docker Compose **junta** y Kubernetes **mantiene**. Antes de empaquetar un programa hay que saber qué necesita para funcionar: tu API necesita Python, Flask y un puerto libre.

Referencias

- Práctica: `github.com/rgap/rgap-Integracion-de-Servicios-Cloud-Gcp-Agosto2026`, carpeta `Sesion27/practica_cafe_nube/`.
- Flask: `flask.palletsprojects.com`. Python y entornos virtuales: `docs.python.org/3/library/venv.html`.
- Docker y Docker Compose: `docs.docker.com`. Kubernetes: `kubernetes.io`.
- Google Cloud: Google Kubernetes Engine (GKE): `cloud.google.com`.
- Material previo del curso: Sesión 11 (Python, entorno virtual y variables de entorno) y Sesión 26 (balanceo de carga, imagen y contenedor).
- Las respuestas de las preguntas están en `../respuestas.md`.